Vigenère-Squad — une équipe d'agents LLM locaux pour la cryptanalyse classique

Vyuob

Version 1.0.0

Table des matières

Résumé / Abstract **2**

1. Problème et motivation **2**

2. Architecture **2**
2.2 Protocole de messages . . . 3

3. Outils **4**
3.1 Catalogue d'outils et listes blanches . . . 4

4. Protocole expérimental **5**

5. Résultats **5**
5.1 Baseline : outils seuls . . . 5
5.2 Agents + outils (politique à règles, sans LLM) . . . 6
5.3 Avec LLM local . . . 8
5.4 Robustesse et pannes . . . 9
5.5 Kasiski contre l'autoclave et distance d'unicité . . . 10
5.6 Étude de cas : une trace complète . . . 11

6. Discussion et limites **13**
6.1 Travaux liés . . . 13
6.2 Enseignements sur les LLM locaux . . . 13
6.3 Perspectives . . . 14
6.4 Usage responsable . . . 14

7. Sécurité **14**

8. Reproductibilité **14**

9. Usage de l'IA **14**

Annexe A — Prompts des agents (version 1.0) **14**
A.1 Prompt du classifieur . . . 14
A.2 Prompt du cryptanalyste . . . 15
A.3 Prompt du linguiste . . . 15

A.4 Prompt du pedagogue 16

Annexe B — Configuration de l'expérience **16**

Annexe C — Reproduire pas à pas **17**

Annexe D — Glossaire **17**

Références **18**

Résumé / Abstract

FR. Vigenère-Squad est un système multi-agents qui identifie et casse six chiffrements classiques (César, affine, Vigenère, Beaufort, autoclave, substitution) sur du texte français. Quatre agents (Classifieur, Cryptanalyste, Linguiste, Pédagogue) pilotés par un orchestrateur explicite appellent des outils déterministes ; le LLM décide et explique, le code calcule. Sur 150 textes × 3 répétitions, les agents à règles (sans LLM) retrouvent le texte dans 85.6 % des cas, contre 86.7 % pour la baseline sans agents ; les textes de 500 lettres ou plus sont tous cassés. Les agents n'améliorent pas la précision ; ils apportent traçabilité, explication et tolérance aux pannes.

EN. Vigenère-Squad is a multi-agent system that identifies and breaks six classical ciphers on French text. Four agents (Classifier, Cryptanalyst, Linguist, Teacher) coordinated by an explicit orchestrator call deterministic tools; the LLM decides and explains, the code computes. On 150 texts × 3 repetitions the rule-based agents recover 85.6 % of the plaintexts versus 86.7 % for a no-agent baseline; all texts of 500+ letters are broken. Agents do not improve accuracy; they add traceability, explanation and fault tolerance.

1. Problème et motivation

Casser un chiffrement classique est une suite de décisions : de quel chiffrement s'agit-il, quelle attaque tenter, le résultat est-il du français, comment l'expliquer ? Les calculs sont mécaniques, le choix de la stratégie l'est moins. C'est le terrain naturel d'un système d'agents, à condition de ne pas confier au LLM ce qu'un programme fait exactement : compter des lettres, tester 312 clés affines, optimiser une permutation.

2. Architecture

Quatre agents à rôles disjoints, un orchestrateur Python explicite (`orchestrator.py`) et un état partagé typé (`EtatPartage`) :

1. **Classifieur** — outils : `indice_coincidence`, `frequences`, `kasiski`. Sortie : famille et ordre des hypothèses.
2. **Cryptanalyste** — outils d'attaque (`chi2_cesar`, `force_brute_affine`, `attaque_vigenere`, `attaque_autoclave`, `attaque_substitution`) et d'analyse (`kasiski`, `friedman`, `ic_colonnes`).
3. **Linguiste** — outil : `score_quadrigrammes` (vraisemblance française et seuil). Valide ou rejette le candidat.
4. **Pédagogue** — outil : `generer_exercice`. Rédige l'explication et propose un exercice

vérifiable par le code.

Boucle de vérification. Si le Linguiste rejette le texte candidat, l'orchestrateur passe à l'hypothèse suivante (6 tentatives au plus). Le verdict final est celui du code (seuil) : un désaccord LLM/code est compté et tracé. **Listes d'outils fermées.** Un outil hors liste, ou des arguments hors bornes, sont refusés avant exécution. **Trace.** Chaque message LLM, appel d'outil, refus, panne et décision est écrit en JSONL. **Le LLM ne recopie jamais le texte** : les outils reçoivent le chiffré ou le candidat par liaison côté code, et le LLM ne voit que des statistiques et un aperçu.

2.2 Protocole de messages

À chaque pas, l'orchestrateur envoie à l'agent **un seul objet JSON** : son rôle, un contexte restreint (hypothèse courante, hypothèses déjà rejetées, graine), la liste des outils qu'il a le droit d'appeler avec leurs arguments, les observations déjà obtenues et une consigne de format. Le texte chiffré n'y figure jamais. L'agent répond par un objet JSON d'une des deux formes `{"action": "tool", "tool": ..., "args": {...}}` ou `{"action": "final", ...}`. La réponse est validée (Pydantic) ; un JSON invalide, une action inconnue ou un outil hors liste sont comptés, tracés et renvoyés à l'agent comme observation d'erreur (3 erreurs de format ou 6 erreurs au total, puis repli de l'orchestrateur).

Avec Ollama, la réponse est de plus **contrainte par un schéma JSON** (sorties structurées) : la clé `action` est obligatoire et limitée à `tool` ou `final`. Ce schéma a été ajouté après un premier essai où `qwen3:4b` recopiait le message reçu au lieu d'y répondre (voir §6) :

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": [
        "tool",
        "final"
      ]
    },
    "tool": {
      "type": "string"
    },
    "args": {
      "type": "object"
    },
    "famille": {
      "type": "string"
    },
    "hypotheses": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
```

```
    "justification": {
      "type": "string"
    },
    "commentaire": {
      "type": "string"
    },
    "plausible": {
      "type": "boolean"
    },
    "raison": {
      "type": "string"
    },
    "explication": {
      "type": "string"
    },
    "exercice": {
      "type": "string"
    }
  },
  "required": [
    "action"
  ]
}
```

Si le LLM est injoignable ou dépasse le délai (300 s par appel, 60 s en mode rapide), l'agent échoue proprement (événement `llm_indisponible` dans la trace) et l'orchestrateur se replie ; il ne plante jamais.

3. Outils

- **Indice de coïncidence** $IC = \sum n_i(n_i - 1)/N(N - 1)$: ≈ 0,078 en français monoalphabétique, ≈ 0,038 pour un texte aléatoire.
- **Kasiski** : les distances entre répétitions de trigrammes sont des multiples de la longueur de clé ; vote des diviseurs.
- **Friedman** : $L \approx (k_p - k_r)N/((N-1)IC - k_rN + k_p)$ avec $k_p = 0{,}0778$, $k_r = 1/26$.
- **Chi²** sur les fréquences du français : décalage de César, et par colonne pour Vigenère. **Beaufort** se ramène à Vigenère par la symétrie $25 - C$.
- **Autoclave** : Kasiski échoue (le flux de clé contient le clair) ; ascension de colline sur le mot-clé, scorée par quadrigrammes.
- **Substitution** : recuit simulé / ascension de colline sur une permutation, avec 6 redémarrages, scorée par un modèle de langue 1–4-grammes interpolé (table 26^4 en $\log_{10}$).

3.1 Catalogue d'outils et listes blanches

Chaque agent ne peut appeler que les outils de sa liste ; les arguments sont validés (type, bornes, valeurs permises) **avant** exécution. Aucun outil n'accède au système de fichiers, au shell ni au

réseau.

outil	genre	agents autorisés	arguments (bornes)	rôle
`indice_coincidence`	analyse	classifieur	—	IC du texte chiffré.
`frequences`	analyse	classifieur	top : [1, 26]	Lettres les plus fréquentes.
`kasiski`	analyse	classifieur, cryptanalyste	max_len : [2, 40]	Répétitions de trigrammes -> longueurs de clé candidates.
`friedman`	analyse	cryptanalyste	—	Estimation de la longueur de clé (Friedman).
`ic_colonnes`	analyse	cryptanalyste	longueur : [1, 40]	IC moyen des colonnes pour une longueur donnée.
`chi2_cesar`	attaque	cryptanalyste	—	Attaque César par chi² sur fréquences françaises.
`force_brute_affine`	attaque	cryptanalyste	—	Essai des 312 clés affines.
`attaque_vigenere`	attaque	cryptanalyste	longueur : [1, 40], variante : [‘vigenere’, ‘beaufort’]	Vigenère/Beaufort : chi² par colonne, longueur donnée.
`attaque_autoclave`	attaque	cryptanalyste	max_cle : [1, 12]	Autokey : ascension de colline sur mot-clé.
`attaque_substitution`	attaque	cryptanalyste	iterations : [100, 8000], redemarrages : [1, 10], seed : [0, 1000000]	Substitution : recuit simulé sur quadrigrammes.
`score_quadrigrammes`	verif	linguiste	—	Score de vraisemblance française + seuil.
`generer_exercice`	exercice	pedagogue	type_chiffre : [‘cesar’, ‘affine’, ‘vigenere’, ‘beaufort’, ‘autoclave’, ‘substitution’], graine : [0, 1000000]	Nouvel exercice du même type (énoncé + corrigé).

4. Protocole expérimental

Corpus généré avec la graine 42 (SHA-256 `b5b1280ccafdb132…`) : 6 chiffrements × 5 longueurs × 5 textes. Les textes clairs sont tirés d’un fichier de test que le modèle de langue n’a jamais vu. Chaque mesure est répétée 3 fois (graines 42–44). Succès : au moins 95 % de lettres exactes. Comparaisons : (a) outils seuls ; (b) agents à règles sans LLM ; (c) un LLM local (`qwen3:4b`) sur un petit sous-ensemble. Tous les paramètres sont dans `config/experiment.yaml` ; l’environnement (versions, matériel, digest des modèles, hachages des prompts) dans `results/ENVIRONMENT.json`. Environnement de cette exécution : Python 3.13.18, Linux-6.18.44-fc-v338-x86_64-with-glibc2.39, 2 CPU, durée 270.9 s, pic mémoire 62.8 Mo. Étape LLM exécutée sur : Windows-10-10.0.26300-SP0, Python 3.11.18, 14 CPU, 15.4 Go de RAM.

5. Résultats

5.1 Baseline : outils seuls

chiffrement	50	100	200	500	1000	global
affine	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
autoclave	20.0 %	100.0 %	100.0 %	100.0 %	100.0 %	84.0 %
beaufort	40.0 %	80.0 %	100.0 %	100.0 %	100.0 %	84.0 %
cesar	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
substitution	0.0 %	13.3 %	66.7 %	100.0 %	100.0 %	56.0 %
vigenere	80.0 %	100.0 %	100.0 %	100.0 %	100.0 %	96.0 %
toutes	56.7 %	82.2 %	94.4 %	100.0 %	100.0 %	**86.7 %**

(σ entre répétitions : 0.0094.)

5.2 Agents + outils (politique à règles, sans LLM)

chiffrement	50	100	200	500	1000	global
affine	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
autoclave	80.0 %	100.0 %	100.0 %	100.0 %	100.0 %	96.0 %
beaufort	20.0 %	40.0 %	100.0 %	100.0 %	100.0 %	72.0 %
cesar	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %	100.0 %
substitution	6.7 %	26.7 %	93.3 %	100.0 %	100.0 %	65.3 %
vigenere	60.0 %	60.0 %	80.0 %	100.0 %	100.0 %	80.0 %
toutes	61.1 %	71.1 %	95.6 %	100.0 %	100.0 %	**85.6 %**

(σ entre répétitions : 0.0031 ; 2.0533 hypothèses testées en moyenne ; 16.28 messages agent par texte ; 0.1589 s par texte.)

Synthèse des trois systèmes.

système	succès	exactitude moyenne	durée moyenne par texte	messages agent par texte
(a) outils seuls	86.7 %	91.6 %	0.1599 s	—
(b) agents à règles	85.6 %	89.9 %	0.1589 s	16.28
(c) LLM `qwen3:4b` (n=6)	50.0 %	51.7 %	335.1783 s	32.6667
(c) LLM `qwen3:0.6b` (n=6)	0.0 %	0.0 %	101.0917 s	26.0
(c) LLM `llama3.2:3b` (n=6)	0.0 %	0.0 %	299.915 s	50.6667

Longueur minimale de succès à 90 % (plus petite longueur testée à partir de laquelle le taux de clair exact est ≥ 90 % pour toutes les longueurs supérieures ; valeurs dérivées de `par_cellule` dans `results/metrics.json`) :

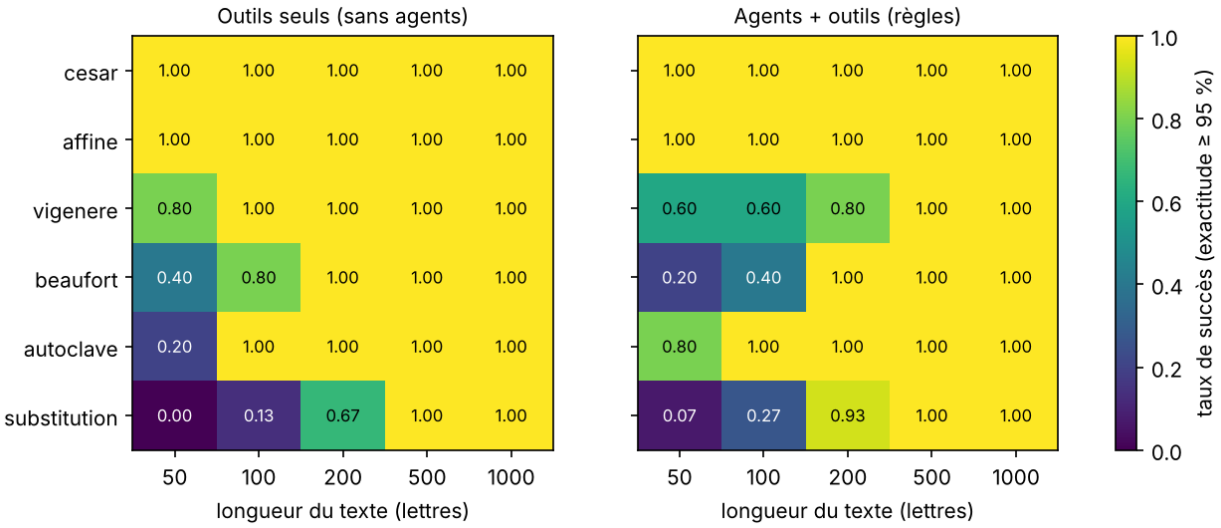


Figure 1: Succès par chiffrement et longueur

chiffrement	outils seuls	agents à règles
affine	50 lettres	50 lettres
autoclave	100 lettres	100 lettres
beaufort	200 lettres	200 lettres
cesar	50 lettres	50 lettres
substitution	500 lettres	200 lettres
vigenere	100 lettres	500 lettres

Lecture. Au-delà de 500 lettres tout est cassé. Les chiffrements monoalphabétiques simples le sont dès 50 lettres. La substitution demande environ 200 lettres ou plus, ce qui est cohérent avec la distance d'unicité (§5.5). Les agents à règles font **moins bien** que la baseline sur Vigenère et Beaufort courts : choisir la longueur de clé par Kasiski + IC des colonnes est moins fiable que tester les longueurs 2–12 et garder le meilleur score.

Cellules où les agents diffèrent de la baseline (écart > 5 points) :

cellule (chiffrement / longueur)	outils seuls	agents à règles	écart
vigenere / 50	80.0 %	60.0 %	-20.0 pts
vigenere / 100	100.0 %	60.0 %	-40.0 pts
vigenere / 200	100.0 %	80.0 %	-20.0 pts
beaufort / 50	40.0 %	20.0 %	-20.0 pts
beaufort / 100	80.0 %	40.0 %	-40.0 pts
autoclave / 50	20.0 %	80.0 %	+60.0 pts
substitution / 50	0.0 %	6.7 %	+6.7 pts
substitution / 100	13.3 %	26.7 %	+13.3 pts
substitution / 200	66.7 %	93.3 %	+26.7 pts

5.3 Avec LLM local

qwen3_4b (`qwen3:4b`) : succès 50.0 %, n=6, σ répétitions sans objet : 1 répétition, appels LLM moyens 32.6667, formats invalides moyens 0.0, durée moyenne 335.1783 s par texte.

qwen3_0.6b (`qwen3:0.6b`) : succès 0.0 %, n=6, σ répétitions sans objet : 1 répétition, appels LLM moyens 26.0, formats invalides moyens 25.0, durée moyenne 101.0917 s par texte.

llama3.2_3b (`llama3.2:3b`) : succès 0.0 %, n=6, σ répétitions sans objet : 1 répétition, appels LLM moyens 50.6667, formats invalides moyens 0.1667, durée moyenne 299.915 s par texte.

Sur 6 textes (un par chiffrement, 200 lettres, 1 répétition), `qwen3:4b` retrouve le clair pour : affine, cesar, substitution ; échec pour : autoclave, beaufort, vigenere. Les réponses sont contraintes par un schéma JSON (sorties structurées d'Ollama) : 0.0 format invalide en moyenne. L'orchestrateur rejette les conclusions non fondées (repli moyen : 2.3333 par texte). **n est très petit : ces chiffres sont indicatifs, pas statistiquement significatifs.** Le coût est élevé (≈ 32.6667 appels et 335.1783 s par texte sur CPU) contre une fraction de seconde pour les agents à règles.

Sur 6 textes (un par chiffrement, 200 lettres, 1 répétition), `qwen3:0.6b` retrouve le clair pour : aucun ; échec pour : affine, autoclave, beaufort, cesar, substitution, vigenere. Les réponses sont contraintes par un schéma JSON (sorties structurées d'Ollama) : 25.0 format invalide en moyenne. L'orchestrateur rejette les conclusions non fondées (repli moyen : 6.0 par texte). **n est très petit : ces chiffres sont indicatifs, pas statistiquement significatifs.** Le coût est élevé (≈ 26.0 appels et 101.0917 s par texte sur CPU) contre une fraction de seconde pour les agents à règles.

Sur 6 textes (un par chiffrement, 200 lettres, 1 répétition), `llama3.2:3b` retrouve le clair pour : aucun ; échec pour : affine, autoclave, beaufort, cesar, substitution, vigenere. Les réponses sont contraintes par un schéma JSON (sorties structurées d'Ollama) : 0.1667 format invalide en moyenne. L'orchestrateur rejette les conclusions non fondées (repli moyen : 6.1667 par texte). **n est très petit : ces chiffres sont indicatifs, pas statistiquement significatifs.** Le coût est élevé (≈ 50.6667 appels et 299.915 s par texte sur CPU) contre une fraction de seconde pour les agents à règles.

chiffrement (200 lettres)	`qwen3:4b`
cesar	réussi (100.0 %)
affine	réussi (100.0 %)
vigenere	échec (0.0 %)
beaufort	échec (0.0 %)
autoclave	échec (0.0 %)
substitution	réussi (100.0 %)

chiffrement (200 lettres)	`qwen3:0.6b`
cesar	échec (0.0 %)
affine	échec (0.0 %)
vigenere	échec (0.0 %)
beaufort	échec (0.0 %)
autoclave	échec (0.0 %)
substitution	échec (0.0 %)

chiffrement (200 lettres)	`llama3.2:3b`
cesar	échec (0.0 %)
affine	échec (0.0 %)
vigenere	échec (0.0 %)
beaufort	échec (0.0 %)
autoclave	échec (0.0 %)
substitution	échec (0.0 %)

Étude d'ablation (textes de 200 lettres, mêmes chiffrements). (a) outils seuls : 94.4 % ; (b) agents à règles, sans LLM : 95.6 % ; (c) agents avec LLM : `qwen3:4b` : 50.0 % (n=6, repli moyen 2.3333) ; `qwen3:0.6b` : 0.0 % (n=6, repli moyen 6.0) ; `llama3.2:3b` : 0.0 % (n=6, repli moyen 6.1667).

Conclusion sur l'apport mesuré des agents. Dans nos mesures, les agents n'apportent **aucun gain de précision significatif** sur la baseline codée en dur (écart d'environ un point, dans les deux sens selon la longueur), et les modèles de langage testés (0,6 à 4 milliards de paramètres, sur CPU) **dégradent** le résultat : seul `qwen3:4b` réussit une partie des textes, les deux autres modèles échouent sur les six. L'apport des agents est ailleurs : traçabilité de chaque décision, explication rédigée, et tolérance aux pannes mesurée (§5.4). Ces mesures LLM portent sur 6 textes et 1 répétition par modèle, sans écart type : elles sont indicatives, pas statistiquement significatives.

5.4 Robustesse et pannes

- 5 % de lettres du chiffré remplacées : succès 48.7 %.
- Clair anglais (le Linguiste est entraîné sur le français) : succès 55.6 %.

Détail du cas « 5 % de bruit » (agents à règles) :

chiffrement	50	100	200	500	1000	global
affine	100.0 %	100.0 %	100.0 %	100.0 %	20.0 %	84.0 %
autoclave	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %
beaufort	0.0 %	40.0 %	80.0 %	100.0 %	20.0 %	48.0 %
cesar	100.0 %	100.0 %	100.0 %	100.0 %	0.0 %	80.0 %
substitution	0.0 %	20.0 %	40.0 %	100.0 %	20.0 %	36.0 %
vigenere	40.0 %	20.0 %	60.0 %	100.0 %	0.0 %	44.0 %
toutes	40.0 %	46.7 %	63.3 %	83.3 %	10.0 %	**48.7 %**

Détail du cas « clair anglais » (agents à règles ; l'ensemble d'entraînement du Linguiste est français) :

chiffrement	200	500	global
affine	100.0 %	100.0 %	100.0 %
autoclave	100.0 %	100.0 %	100.0 %
beaufort	0.0 %	0.0 %	0.0 %
cesar	66.7 %	100.0 %	83.3 %
substitution	0.0 %	0.0 %	0.0 %

chiffrement	200	500	global
vigenere	33.3 %	66.7 %	50.0 %
toutes	50.0 %	61.1 %	**55.6 %**

Injection de pannes :

taux de pannes	succès (LLM corrompu)	repli moyen	succès (outils en panne)
0.0	98.3 %	0.0	98.3 %
0.1	98.3 %	0.0333	98.3 %
0.2	93.3 %	0.25	98.3 %
0.3	85.0 %	0.85	98.3 %

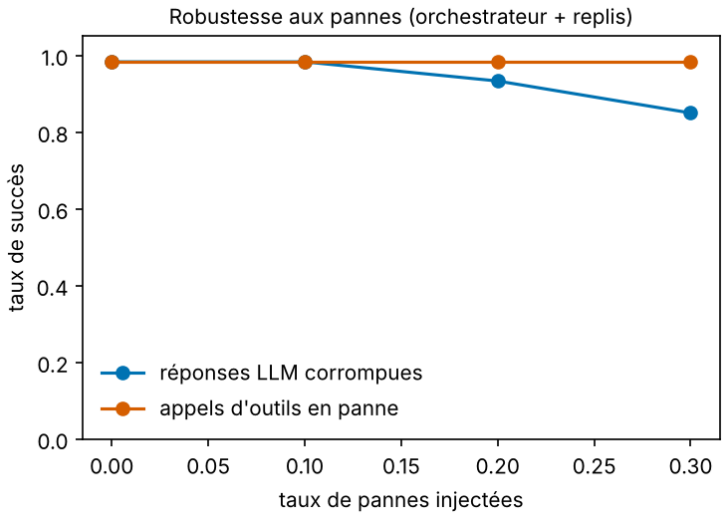


Figure 2: Robustesse aux pannes

Les pannes d'outils sont absorbées par de nouvelles tentatives dans l'agent. Les réponses LLM corrompues (JSON invalide, action inconnue, vide) déclenchent un rejet de schéma puis, au-delà de 3 erreurs, un repli de l'orchestrateur.

5.5 Kasiski contre l'autoclave et distance d'unicité

La vraie longueur de clé figure dans le top 3 de Kasiski pour 100.0 % des Vigenère (n=20) contre 65.0 % des autoclaves (n=20) : l'autoclave brouille Kasiski. Le modèle de langue donne une entropie de 3.127 bits/lettre (borne haute), soit une redondance de 1.573 bits/lettre et des distances d'unicité $U = H(K)/D$ de 3.0 lettres (César), 5.3 (affine), 17.9 (Vigenère, clé de 6) et 56.2 (substitution) : ordres de grandeur, car l'entropie est surestimée sur un petit corpus. Le succès réel exige davantage de lettres que U : l'unicité est une condition nécessaire, pas suffisante, pour un attaquant au calcul borné.

Le Pédagogue produit des exercices valides dans 100.0 % des cas (n=60, vérifiés par déchiffrement avec la clé du corrigé).

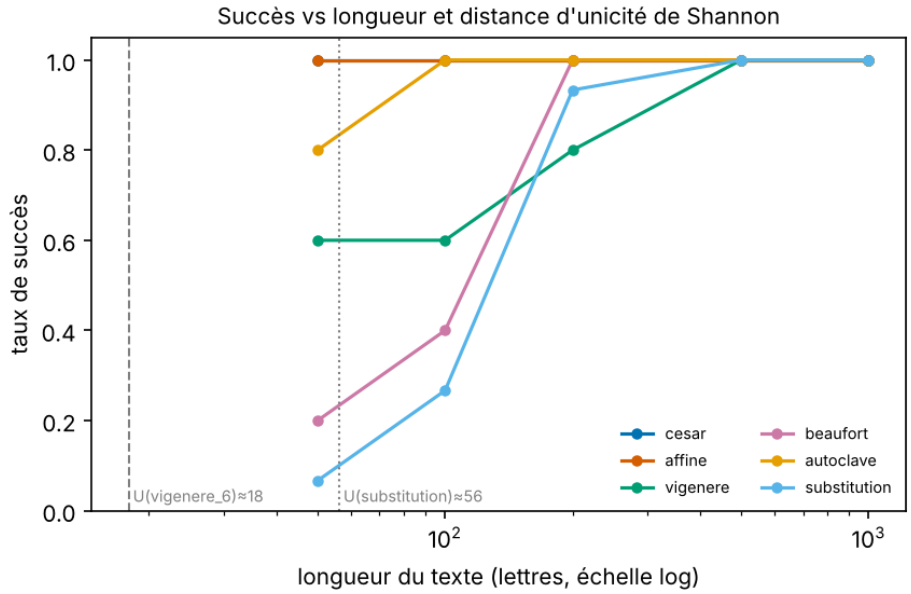


Figure 3: Succès vs longueur

5.6 Étude de cas : une trace complète

Trace réelle d'un texte chiffré par Vigenère (200 lettres, clé de 5), résolu par les agents à règles. Les « réponses » des agents sont ici produites par la politique à règles, selon le même protocole JSON que le LLM : l'orchestrateur ne voit pas la différence.

#	t (ms)	agent	type	contenu
0	0	orchestrateur	debut	{'longueur': 200}
1	0	classifieur	llm	réponse : `{"action": "tool", "tool": "indice_coincidence", "args": {}}`
2	1	classifieur	outil	outil `indice_coincidence` {} → {'ic': 0.0469, 'n': 200, 'ref_fr': 0.0778, 'ref_aleatoire': 0.0385}
3	1	classifieur	llm	réponse : `{"action": "tool", "tool": "kasiski", "args": {}}`
4	1	classifieur	outil	outil `kasiski` {} → {'n_repetitions': 7, 'longueurs_candidates': [{'longueur': 5, 'votes'…
5	1	classifieur	llm	réponse : `{"action": "final", "famille": "polyalphabetique", "hypotheses": ["vigenere", "…`
6	1	orchestrateur	decision	{'etape': 'classification', 'famille': 'polyalphabetique', 'hypotheses': ['vigenere', 'beaufort', 'autoclave'…
7	1	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "kasiski", "args": {"max_len": 12}}`
8	1	cryptanalyste	outil	outil `kasiski` {'max_len': 12} → {'n_repetitions': 7, 'longueurs_candidates': [{'longueur': 5, 'votes'…
9	1	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "ic_colonnes", "args": {"longueur": 2}}`
10	2	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 2} → {'longueur': 2, 'ic_moyen': 0.048}
11	2	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "ic_colonnes", "args": {"longueur": 3}}`
12	2	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 3} → {'longueur': 3, 'ic_moyen': 0.0479}
13	2	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "ic_colonnes", "args": {"longueur": 5}}`
14	2	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 5} → {'longueur': 5, 'ic_moyen': 0.0849}
15	2	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "ic_colonnes", "args": {"longueur": 10}}`
16	2	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 10} → {'longueur': 10, 'ic_moyen': 0.0837}
17	2	cryptanalyste	llm	réponse : `{"action": "tool", "tool": "attaque_vigenere", "args": {"longueur": 5, "variant…`
18	4	cryptanalyste	outil	outil `attaque_vigenere` {'longueur': 5, 'variante': 'vigenere'} → {'cle': 'XMSLG', 'clair_apercu': 'ENTEURSACHERCHERDESMETHODESOUCHACUN…
19	5	cryptanalyste	llm	réponse : `{"action": "final", "tool": "attaque_vigenere", "commentaire": "longueur 5, var…`

#	t (ms)	agent	type	contenu
20	6	linguiste	llm	réponse : `{"action": "tool", "tool": "score_quadrigrammes", "args": {}}`
21	8	linguiste	outil	outil `score_quadrigrammes` {} → {'score': -0.8993, 'seuil': -1.6413, 'au_dessus_seuil': True, 'apercu…
22	8	linguiste	llm	réponse : `{"action": "final", "plausible": true, "raison": "score -0.8993 > seuil -1.6413…`
23	9	orchestrateur	decision	{'etape': 'verification', 'hypothese': 'vigenere', 'score': -0.899, 'accepte': True}
24	9	pedagogue	llm	réponse : `{"action": "tool", "tool": "generer_exercice", "args": {"type_chiffre": "vigene…`
25	9	pedagogue	outil	outil `generer_exercice` {'type_chiffre': 'vigenere', 'graine': … → {'type': 'vigenere', 'enonce': 'RGREXELYCKRKAHHEWCSPRMFOGRLMUFUTAUQCH…
26	9	pedagogue	llm	réponse : `{"action": "final", "explication": "Le texte a été identifié comme un chiffreme…`
27	9	orchestrateur	fin	{'statut': 'resolu', 'chiffrement': 'vigenere', 'duree_s': 0.01}

On y lit : (1) le Classifieur mesure l'IC (0,047, proche de l'aléatoire) et Kasiski (7 répétitions) et conclut « polyalphabétique » ; (2) le Cryptanalyste teste les longueurs candidates et retient celle dont l'IC par colonne est élevé ; (3) le Linguiste compare le score de quadrigrammes au seuil ; (4) l'orchestrateur accepte ; (5) le Pédagogue génère un exercice. Chaque étape est rejouable avec `vigenere-squad rejouer <trace>`.

Trace avec un vrai LLM (`qwen3:4b`, Beaufort, 200 lettres, début de la trace) :

#	t (ms)	agent	type	contenu
0	7	orchestrateur	debut	{'longueur': 200}
1	5225	classifieur	llm	réponse : `{ "action": "tool", "tool": "indice_coincidence", "args": {} }`
2	5227	classifieur	outil	outil `indice_coincidence` {} → {'ic': 0.0435, 'n': 200, 'ref_fr': 0.0778, 'ref_aleatoire': 0.0385}
3	13194	classifieur	llm	réponse : `{ "action": "tool", "tool": "frequences", "args": { "top": 5 } }`
4	13200	classifieur	outil	outil `frequences` {'top': 5} → {'top': {'Z': 0.08, 'J': 0.075, 'T': 0.07, 'D': 0.07, 'B': 0.065}}
5	38258	classifieur	llm	réponse : `{ "action": "final", "famille": "polyalphabetique", "hypotheses": [ "…`
6	38260	orchestrateur	decision	{'etape': 'classification', 'famille': 'polyalphabetique', 'hypotheses': ['vigenere', 'beaufort', 'autoclave'…
7	52706	cryptanalyste	llm	réponse : `{ "action": "tool", "tool": "friedman", "args": {} }`
8	52708	cryptanalyste	outil	outil `friedman` {} → {'longueur_estimee': 7.53}
9	60384	cryptanalyste	llm	réponse : `{ "action": "tool", "tool": "kasiski", "args": { "max_len": 40 } }`
10	60386	cryptanalyste	outil	outil `kasiski` {'max_len': 40} → {'n_repetitions': 3, 'longueurs_candidates': [{'longueur': 2, 'votes'…
11	69384	cryptanalyste	llm	réponse : `{ "action": "tool", "tool": "ic_colonnes", "args": { "longueur": 8 …`
12	69387	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 8} → {'longueur': 8, 'ic_moyen': 0.0733}
13	77117	cryptanalyste	llm	réponse : `{ "action": "tool", "tool": "ic_colonnes", "args": { "longueur": 8 …`
14	77119	cryptanalyste	outil	outil `ic_colonnes` {'longueur': 8} → {'longueur': 8, 'ic_moyen': 0.0733}
15	90097	cryptanalyste	llm	réponse : `{ "action": "final", "tool": "attaque_vigenere", "commentaire": "La longu…`
16	90098	orchestrateur	repli	{'cible': 'cryptanalyste', 'hypothese': 'vigenere', 'action': 'hypothèse ignorée'}
17	106518	cryptanalyste	llm	réponse : `{ "action": "tool", "tool": "friedman", "args": {} }`

On y voit le modèle enchaîner des appels d'outils valides, puis conclure « final » en citant une attaque qu'il n'a jamais exécutée : l'orchestrateur le détecte et ignore l'hypothèse (événement `repli`). C'est le comportement attendu de la séparation « le LLM décide, le code vérifie ».

6. Discussion et limites

- **Résultat négatif assumé** : les agents ne battent pas la baseline en précision. Leur valeur : modularité, trace complète, explications, tolérance aux pannes, listes d'outils fermées.
- **Corpus d'entraînement minuscule** (≈ 6,7 ko) : modèle de langue pauvre, entropie surestimée, textes de test qui se recouvrent aux grandes longueurs.
- **Linguiste** : le seuil accepte parfois un clair en partie faux ; il suppose le français.
- **LLM** : évalué sur un sous-ensemble minimal (6 textes de 200 lettres, 1 répétition, trois modèles : `qwen3:4b`, `qwen3:0.6b` et `llama3.2:3b`) pour tenir sur un portable sans GPU. Les mesures LLM sont donc indicatives.
- Pas de comparaison avec un LLM seul sans outils (hors périmètre) ; les LLM de ≤ 8 Go calculent mal des fréquences, ce qui justifie la répartition « LLM décide / code calcule ».

6.1 Travaux liés

Les outils reprennent des méthodes classiques : examen de Kasiski (1863) pour la longueur de clé, indice de coïncidence et test de Friedman (1922), distance d'unicité de Shannon (1949), recuit simulé de Kirkpatrick et al. (1983) pour la substitution. La nouveauté ne se situe pas dans ces algorithmes, mais dans leur **orchestration** : des agents à rôles disjoints, des outils à listes fermées, une trace complète et un contrôle par le code des conclusions du LLM.

6.2 Enseignements sur les LLM locaux

- Un petit modèle (0,6 milliard de paramètres) ne suit pas le protocole de façon fiable ; un modèle de 4 milliards le suit si la sortie est contrainte par un schéma, mais raisonne mal sur les chiffrements polyalphabétiques.
- Sans schéma, `qwen3:4b` a recopié le message reçu au lieu d'y répondre : chaque réponse invalide coûtait environ 60 s sur CPU et le test devenait impraticable. Les sorties structurées d'Ollama règlent le problème de format, pas celui du raisonnement.
- Un délai maximal par appel et un repli propre sont indispensables : sur un processeur partagé (CI), un appel peut dépasser 60 s.
- **Boucles** : sur les traces de `qwen3:4b`, le Cryptanalyste rappelle parfois le même outil (jusqu'à 13 fois de suite `ic_colonnes` avec la même longueur) jusqu'à la limite de 14 pas, ce qui déclenche le repli ; l'outil correct (par exemple `force_brute_affine` pour le texte affine) a parfois été appelé trop tard.
- **Explications peu fiables** : le Pédagogue produit des phrases fausses (« clé de 20 caractères » pour une permutation de 26 lettres, « substitution polyalphabétique »), et n'appelle pas l'outil `generer_exercice` : l'exercice proposé n'est alors pas vérifié par le code, contrairement à la politique à règles (100 % d'exercices valides). C'est une limite connue de la version LLM.
- **Format des hypothèses** : dans un cas, le Classifieur a recopié le gabarit du prompt (`monoalphabetique|polyalphabetique`) au lieu de choisir une famille.
- Le coût est très asymétrique : environ 335 s par texte avec le LLM contre une fraction de seconde pour les règles, pour une précision inférieure.

6.3 Perspectives

Évaluer plus de textes et de répétitions avec le LLM ; tester d'autres modèles locaux et des tailles de contexte différentes ; élargir le corpus d'entraînement du Linguiste (plusieurs langues) ; remplacer la politique à règles du Cryptanalyste par un LLM guidé par des exemples (few-shot) ; ajouter des chiffrements (Playfair, Hill, transposition).

6.4 Usage responsable

Le système ne casse que des chiffrements historiques, sans valeur de sécurité aujourd'hui, sur des textes de test. Il n'envoie aucune donnée hors de la machine locale. Les exercices générés servent à l'apprentissage de la cryptanalyse.

7. Sécurité

Modèle de menaces STRIDE complet dans `docs/THREAT_MODEL.md` (table menace / scénario / contre-mesure / test) : injection via le texte chiffré (le texte n'entre pas dans le prompt), clair inventé par le LLM (le code a le dernier mot), boucle infinie (bornes de pas et d'essais), élévation de privilèges (listes blanches, aucun outil shell/fichier/réseau), fuite de données (réseau local uniquement, gitleaks en CI).

8. Reproductibilité

`make reproduce` rejoue tout (tests, corpus, baseline, agents, LLM, métriques, figures, site). `make reproduce-small` (CI) utilise `qwen3:0.6b`. Environnement figé par `uv.lock` et `Dockerfile` ; `SHA256SUMS` signé ; archive Zenodo de la release `v1.0.0`.

9. Usage de l'IA

Voir `AI_USAGE.md` : conception et code réalisés avec l'assistance de Claude (Anthropic), sous la direction et la relecture de l'auteur ; résultats issus uniquement de l'exécution du dépôt.

Annexe A — Prompts des agents (version 1.0)

Les prompts sont versionnés dans `src/vigenere_squad/prompts/` ; leurs empreintes SHA-256 figurent dans `results/ENVIRONMENT.json`.

A.1 Prompt du classifieur

> Tu es le CLASSIFIEUR d une équipe de cryptanalystes. Ton rôle : déterminer la famille du chiffrement d un texte
> et ordonner les hypothèses à tester.
> Repères : l indice de coïncidence (IC) vaut ≈0,078 pour du français chiffré par substitution simple

(César, affine, substitution) et ≈0,038–0,050 pour un chiffrement polyalphabétique (Vigenère, Beaufort, autoclave).
Si peu de répétitions de trigrammes (Kasiski) malgré un IC bas, privilégie « autoclave ».
Hypothèses autorisées : cesar, affine, substitution, vigenere, beaufort, autoclave.
Format final : {“action”:“final”,“famille”:“monoalphabetique|polyalphabetique”,“hypotheses”:[…ordre de test…],“justification”:“une phrase”}
Tu réponds UNIQUEMENT par un objet JSON. Deux formes possibles :
- appel d outil : {“action”:“tool”,“tool”:“”,“args”:{…}}
- réponse finale : voir le format de ton rôle.
Tu ne peux appeler QUE les outils listés. Le texte à analyser est fourni automatiquement aux outils : ne le recopie jamais. Un outil appelé deux fois avec les mêmes arguments donne le même résultat : n en refais pas l appel. Les nombres viennent des outils, jamais de toi.

A.2 Prompt du cryptanalyste

Tu es le CRYPTANALYSTE. On te donne UNE hypothèse de chiffrement (contexte.hypothese). Lance l attaque adaptée :
cesar -> chi2_cesar ; affine -> force_brute_affine ; substitution -> attaque_substitution ; autoclave -> attaque_autoclave ;
vigenere/beaufort -> kasiski (puis ic_colonnes sur chaque longueur candidate) puis attaque_vigenere(longueur, variante)
avec la PLUS PETITE longueur dont l IC moyen des colonnes est proche du meilleur (≥ 0,06 attendu).
Si kasiski ne donne rien (texte court), essaie les longueurs 2 à 8 avec ic_colonnes.
Format final : {“action”:“final”,“tool”:“”,“commentaire”:“une phrase”}
Le texte déchiffré est récupéré automatiquement depuis le résultat de l outil.
Tu réponds UNIQUEMENT par un objet JSON. Deux formes possibles :
- appel d outil : {“action”:“tool”,“tool”:“”,“args”:{…}}
- réponse finale : voir le format de ton rôle.
Tu ne peux appeler QUE les outils listés. Le texte à analyser est fourni automatiquement aux outils : ne le recopie jamais. Un outil appelé deux fois avec les mêmes arguments donne le même résultat : n en refais pas l appel. Les nombres viennent des outils, jamais de toi.

A.3 Prompt du linguiste

Tu es le LINGUISTE. On te soumet un texte déchiffré candidat. Appelle score_quadrigrammes ; le résultat contient un seuil et
un aperçu des 80 premières lettres. Dis si le texte est du français plausible (mots reconnaissables, score au-dessus du seuil).
Rejette si le score est sous le seuil.
Format final : {“action”:“final”,“plausible”:true|false,“raison”:“une phrase”}
Tu réponds UNIQUEMENT par un objet JSON. Deux formes possibles :
- appel d outil : {“action”:“tool”,“tool”:“”,“args”:{…}}

- réponse finale : voir le format de ton rôle.
Tu ne peux appeler QUE les outils listés. Le texte à analyser est fourni automatiquement aux outils : ne le recopie jamais. Un outil appelé deux fois avec les mêmes arguments donne le même résultat : n en refais pas l appel. Les nombres viennent des outils, jamais de toi.

A.4 Prompt du pedagogue

Tu es le PÉDAGOGUE. À partir du résumé de l enquête (contexte), rédige en français, en 3 phrases maximum, une explication
destinée à un étudiant : comment on a reconnu le chiffrement et comment la clé a été trouvée. Appelle ensuite generer_exercice
(même type de chiffre) pour proposer un exercice. Ne divulgue pas de clé inventée : utilise celle du contexte.
Format final : {“action”:“final”,“explication”:“…”,“exercice”:“”}
Tu réponds UNIQUEMENT par un objet JSON. Deux formes possibles :
- appel d outil : {“action”:“tool”,“tool”:“”,“args”:{…}}
- réponse finale : voir le format de ton rôle.
Tu ne peux appeler QUE les outils listés. Le texte à analyser est fourni automatiquement aux outils : ne le recopie jamais. Un outil appelé deux fois avec les mêmes arguments donne le même résultat : n en refais pas l appel. Les nombres viennent des outils, jamais de toi.

Annexe B — Configuration de l’expérience

```yaml
# Configuration unique de l'expérience : tout paramètre qui influence un résultat
↪ est ici.
seed: 42
corpus:
  langue: fr
  longueurs: [50, 100, 200, 500, 1000]
  par_cellule: 5            # 6 chiffrements x 5 longueurs x 5 = 150 échantillons
  repetitions: 3            # >= 3 répétitions par mesure (graines seed, seed+1,
↪ seed+2)
robustesse:
  bruit_frappe: 0.05        # 5 % de lettres du chiffré remplacées
  anglais: true             # texte clair anglais (le linguiste est entraîné sur le
↪ français)
pannes:
  p_llm: [0.0, 0.1, 0.2, 0.3]   # proportion de réponses LLM corrompues
  p_outil: [0.0, 0.1, 0.2, 0.3] # proportion d'appels d'outil en panne
llm:
  temperature: 0.0
  seed: 42
```

```
  num_predict: 512          # nombre max de jetons générés par réponse (les réponses
↪  JSON attendues sont courtes)
  timeout_s: 300            # délai max par appel LLM ; dépassé -> l'agent échoue
↪  proprement et l'orchestrateur se replie sur les outils
  num_ctx: 4096             # 8 Go : contexte 4096 (cf. annexe A)
  modeles:                  # `digest` est relevé automatiquement via l'API Ollama
↪  et écrit dans results/ENVIRONMENT.json
    principal: {etiquette: "qwen3:4b", ollama: "qwen3:4b"}
    second:    {etiquette: "llama3.2:3b", ollama: "llama3.2:3b"}
  sous_ensemble: {longueurs: [200], par_cellule: 2}   # 6 chiffrements x 1 longueur
↪  x 2 = 12 échantillons
  repetitions: 3
profil_small:                # `make reproduce-small` (CI) : petit modèle, 1
↪  répétition
  modele: "qwen3:0.6b"
  timeout_s: 60
  sous_ensemble: {longueurs: [200], par_cellule: 1}
  repetitions: 1
  taille_corpus: {longueurs: [100, 500], par_cellule: 2}
succes_seuil_exactitude: 0.95
```

Annexe C — Reproduire pas à pas

Linux / macOS (avec `make`) :

```
uv sync --frozen --extra dev
make test
make models && make reproduce-small   # rapide
make reproduce                        # complet (plusieurs heures sur CPU)
```

Windows (sans `make`, PowerShell) :

```
uv sync --frozen --extra dev
uv run --frozen pytest
uv run --frozen python scripts/reproduce.py --small
uv run --frozen python -m vigenere_squad.evaluate --llm-seul --modeles qwen3:4b
↪  --par-cellule 1 --repetitions 1 --timeout 240
```

Pour reproduire uniquement les mesures sans LLM : `uv run --frozen python scripts/reproduce.py --no-llm`. Vérification d'intégrité : `uv run --frozen python scripts/hashes.py --check` puis `minisign -Vm SHA256SUMS -p minisign.pub`.

Annexe D — Glossaire

- **IC** : indice de coïncidence, probabilité que deux lettres tirées au hasard dans le texte soient identiques.
- **Kasiski** : méthode qui déduit la longueur de la clé des distances entre séquences répétées.
- **Autoclave (autokey)** : Vigenère dont la clé se prolonge avec le texte clair lui-même.

- **Beaufort** : variante de Vigenère où l'on soustrait le texte à la clé.
- **Quadrigramme** : suite de quatre lettres ; leur fréquence mesure la vraisemblance d'un texte.
- **Repli** : décision de l'orchestrateur d'abandonner une hypothèse ou un agent défaillant.
- **Trace** : journal JSONL de tous les messages, appels d'outils et décisions.
- **Schéma JSON** : description de la forme exacte attendue pour la réponse d'un LLM.

Références

- Kasiski, F. W. (1863). *Die Geheimschriften und die Dechiffrir-Kunst*.
- Friedman, W. F. (1922). *The Index of Coincidence and Its Applications in Cryptography*.
- Shannon, C. E. (1949). Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4).
- Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220.